

Avaliação de Predição de Desvios no HeapSort em RISC-V com o Simulador gem5

Matheus H. R. da Costa¹, Lucas S. Schuch¹, Kassiara R. Bosing¹,
Jean C. C. Gondorek¹, Jonathan G. Correa¹

¹Universidade Federal da Fronteira Sul (UFFS)
Chapecó – SC – Brasil

{matheus.dacosta, lucas.schuch, kassara.bosing}@estudante.uffs.edu.br

{jean.gondorek, jonathan.correa}@estudante.uffs.edu.br

Abstract. *This work evaluates branch prediction techniques on a HeapSort algorithm implemented in RISC-V assembly and simulated with gem5. As required by the assigned topic, two predictors — LocalBP (local, simple) and BiModeBP (global bi-mode) — are compared on an in-order (TIMING) pipeline, measuring the total number of evaluated branches and misprediction counts extracted from stats.json. With an identical set of evaluated branches, BiModeBP reduces mispredictions by about 38% (and 45% on the data-dependent DirectCond branches of heapify), improving prediction accuracy where purely local predictors struggle.*

Resumo. *Este trabalho avalia técnicas de predição de desvios no HeapSort implementado em assembly RISC-V e simulado no gem5. Conforme o tópico escolhido, dois preditores — LocalBP (local simples) e BiModeBP (bi-mode global) — são comparados em pipeline em-ordem (TIMING), medindo o total de desvios avaliados e a quantidade de predições incorretas extraídas de stats.json. Com o mesmo conjunto de desvios avaliados, o BiModeBP reduz as predições erradas em cerca de 38% (e 45% nos desvios condicionais diretos DirectCond da rotina heapify), melhorando a precisão onde preditores puramente locais têm dificuldade.*

1. Introdução

No Trabalho 1 da disciplina de Organização de Computadores, validou-se a corretude funcional de um HeapSort em RISC-V utilizando simuladores instrucionais (Venus). Neste trabalho, o mesmo algoritmo é analisado sob a ótica de microarquitetura com o simulador de ciclo de clock gem5 [Binkert et al. 2011], conforme especificação da atividade [Curso de Ciência da Computação – UFFS 2026].

O tópico escolhido é **Predição de Desvios** (tópico 2): comparar LocalBP com BiModeBP [Lee and Chang 1997], ambos no modelo *em-ordem* (TIMING), conforme exigido pelo enunciado, medindo o total de desvios avaliados e a quantidade de erros de predição. O HeapSort apresenta laços regulares e desvios dependentes de dados em heapify, sendo um caso de estudo adequado.

2. Fundamentação Teórica

2.1. Predição de desvios

Em pipelines com múltiplos estágios, instruções de desvio condicional exigem que o processador preveja o próximo endereço de execução antes que a condição seja resolvida [Hennessy and Patterson 2017]. Predições incorretas geram *bubbles*: ciclos perdidos enquanto o pipeline é refeito a partir do endereço correto. O **LocalBP** mantém um histórico de desvios por endereço de instrução (PC); é eficiente quando o mesmo PC tem comportamento estável, mas sofre com *aliasing* entre PCs distintos. O **BiModeBP** combina um preditor local com escolha por PC e um preditor global, separando padrões predominantemente taken e not-taken, o que reduz o *aliasing* em código com mistura de laços e comparações.

2.2. Desvios no HeapSort

O HeapSort implementado utiliza *min-heap* e produz ordenação decrescente *in-place*. Os pontos críticos de desvio são:

- **Laços regulares:** `build_loop` e `extract_loop` executam comparações de contador (`blt/ble`) com padrão previsível.
- **Comparações em `heapify`:** instruções `blt` comparam valores do vetor; o resultado depende dos **dados**, dificultando preditores puramente locais.
- **Chamadas recursivas:** `jal/ret` geram desvios de chamada/retorno, geralmente bem previstos pela Return Address Stack (RAS).

3. Metodologia

3.1. Ambiente e compilação

O código assembly `heapSort_original.s` foi adaptado do Trabalho 1 para a sintaxe GCC, com ponto de entrada `main`, uso de `printf` e alocação via `malloc`. A compilação gera binário estático exigido pelo `gem5`:

```
riscv64-linux-gnu-gcc -static heapSort_original.s \
-o heapSort_original.riscv
```

As simulações foram executadas em ambiente Docker com `gem5 (gem5.opt)` e o script `inorder.py`. Todos os cenários usam a CPU `CPUTypes.TIMING` (pipeline em-ordem) com cache L1 de 32 kB, L2 de 256 kB, memória DDR3 de 1 GB e clock de 3 GHz; a comparação entre preditores de desvio (`LocalBP` vs. `BiModeBP`) mantém os demais parâmetros de hardware idênticos entre os dois cenários.

3.2. Geração assistida das simulações

Pedimos a um modelo de LLM que reescrevesse `inorder.py` aceitando, via `argparse`, CPU, preditor de desvio, clock, caches L1/L2, hierarquia, núcleos e memória, parametrização usada para fixar os valores padrão (CPU `TIMING`, 3 GHz, L1 32 kB, L2 256 kB, DDR3 1 GB) na comparação dos preditores. Executamos a comparação com o vetor padrão ($n = 6$) e, para validar o comportamento em escala maior, também com $n = 100.000$ elementos, gerado pela IA em assembly puro a partir de `heapSort_gem5.s`; ambas as execuções usam os mesmos valores padrão do `inorder.py`.

3.3. Cenários experimentais

Dois cenários foram comparados mantendo-se idênticos os demais parâmetros de hardware:

- **Cenário A:** LocalBP — preditor local simples.
- **Cenário B:** BiModeBP — preditor bi-mode global.

O vetor de entrada padrão contém $n = 6$ elementos ($\{12, 11, 13, 5, 6, 7\}$). Complementarmente, executou-se a mesma comparação com $n = 100\,000$ elementos, também na CPU TIMING a 3 GHz, para validar o comportamento em escala maior.

3.4. Métricas coletadas

As estatísticas foram extraídas do arquivo `m5out/stats.json` gerado após cada simulação, com foco nas métricas exigidas pelo enunciado:

- `branchPred.BTBLookups` — total de desvios condicionais avaliados;
- `branchPred.condIncorrect` — predições incorretas;
- `simTicks` — tempo total de simulação;
- IPC, calculado como `simInsts / core_numCycles`.

A taxa de erro global foi calculada como `taxa = condIncorrect/lookups`, ou seja, predições incorretas sobre o total de desvios avaliados. Todas as métricas de `branchPred` (Figuras 1 e 2) e a Tabela 1 usam a CPU em-ordem (TIMING), conforme o tópico 2 do enunciado.

4. Resultados

4.1. Cenário padrão (3 GHz, pipeline em-ordem)

A entrega parcial confirmou a execução correta do binário no gem5. A Tabela 1 resume as métricas do cenário padrão (LocalBP) com CPU em-ordem a 3 GHz.

Tabela 1. Métricas do cenário padrão (CPU em-ordem, 3 GHz, $n = 6$, LocalBP)

Métrica	Valor
<code>simTicks</code>	106.596.297
IPC	0,395
Instruções simuladas (<code>simInsts</code>)	126.371
Misses L1D	548 (taxa 1,95%)

4.2. Comparação LocalBP vs. BiModeBP (pipeline em-ordem)

A Figura 1 apresenta a comparação direta entre os dois preditores na CPU TIMING, com $n = 6$. Como o pipeline em-ordem efetiva os mesmos desvios independentemente do preditor, o total de desvios avaliados (`lookups`) é idêntico nos dois cenários (30.992); o que muda é a quantidade de erros.

O BiModeBP reduziu substancialmente as predições incorretas (**38,1%** a menos que o LocalBP), especialmente nos desvios condicionais diretos (`beq/blt`, **45,4%** a menos), que correspondem às comparações em `heapify` e aos laços de construção e extração do *heap*.

LocalBP vs. BiModeBP — pipeline em-ordem (TIMING, $n=6$)

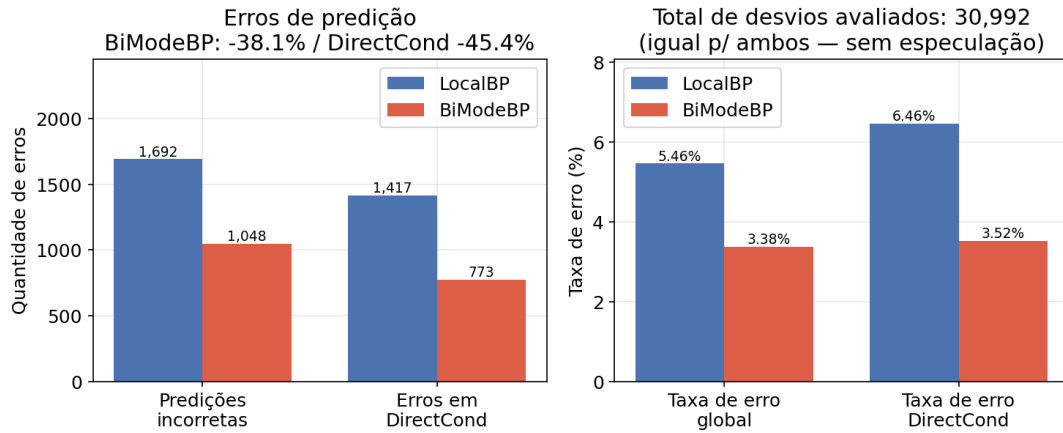


Figura 1. Comparação dos preditores em pipeline em-ordem (TIMING, $n = 6$): quantidade de erros (esq.) e taxas de erro (dir.).

Apesar da redução expressiva nas predições incorretas, o `simTicks` é *idêntico* entre os dois preditores ($\approx 106,6$ milhões): o modelo em-ordem do `gem5` mede corretamente quantos erros ocorrem, mas não aplica penalidade de ciclos por *misprediction* no tempo simulado.

4.3. Efeito de escala ($n=100.000$)

Para checar se o padrão observado em $n = 6$ se mantém em escala maior, repetimos a comparação com $n = 100.000$ na mesma CPU TIMING. A Figura 2 mostra a taxa de erro global nas duas escalas: o BiModeBP reduz a taxa de erro em ambas (**38,1%** em $n = 6$ e **24,9%** em $n = 100.000$), e o `simTicks` permanece idêntico entre preditores também em $n = 100.000$, confirmando que o efeito é específico do modelo em-ordem, não do tamanho da entrada.

5. Discussão

Os laços `build_loop` e `extract_loop` possuem comportamento de desvio estável, sendo bem servidos por ambos os preditores. Já em `heapify`, cada nó executa até duas comparações `blt` cujo resultado depende dos valores no vetor; o mesmo endereço de instrução é reexecutado com resultados distintos a cada chamada recursiva, o que eleva `condIncorrect` no LocalBP. O BiModeBP mitiga esse efeito ao separar históricos taken/not-taken e empregar informação global, reduzindo a taxa de erro em DirectCond de 6,46% para 3,52%.

A medição em-ordem responde diretamente ao tópico (quantos desvios e quantos erros), mas não captura o custo desses erros em tempo: o pipeline TIMING do `gem5` não modela a penalidade de *misprediction*, por isso o `simTicks` é idêntico entre os preditores tanto em $n = 6$ quanto em $n = 100.000$. Conclui-se que, neste modelo, a vantagem do BiModeBP se manifesta em *precisão* (menos erros), não em tempo de execução simulado.

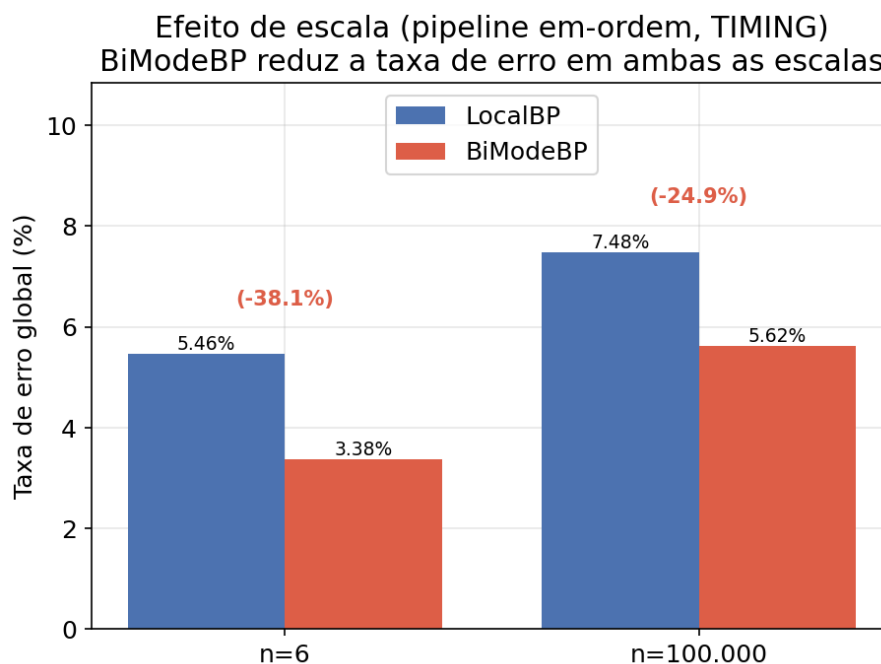


Figura 2. Taxa de erro global ($\text{condIncorrect}/\text{lookups}$) em pipeline em-ordem, $n = 6$ vs. $n = 100.000$.

6. Conclusão

Para o HeapSort em RISC-V simulado no gem5 em pipeline em-ordem, o preditor **BiModeBP** adapta-se melhor que o **LocalBP**. A redução de 38,1% nas previsões incorretas (45,4% nos desvios condicionais diretos) decorre da maior precisão nos desvios de `heapify`, onde comparações dependentes de dados desafiam preditores puramente locais. Esse ganho de precisão se mantém em $n = 100.000$ (redução de 24,9%), mas não se traduz em tempo de simulação no modelo `TIMING`, que não penaliza *mispredictions* em ciclos. Recomenda-se o BiModeBP para cargas com padrão misto de laços regulares e comparações data-dependent; como trabalho futuro, sugere-se repetir a comparação em pipeline fora de ordem para observar o efeito da predição sobre o tempo de execução.

Declaração de uso de IA

Ferramentas de inteligência artificial foram utilizadas na reescrita de `inorder.py` e do HeapSort em escala (`heapSort_100k.s`) e na revisão textual deste relatório, conforme descrito na Seção 3.2. A interpretação dos resultados, a execução dos experimentos e a análise microarquitetural são de responsabilidade dos autores.

Referências

- Binkert, N., Beckmann, B., Black, G., Reinhardt, S. K., Saidi, A., Basu, A., Beck, J., Hower, D. R., Raymond, C. H., Wood, D. A., et al. (2011). The gem5 simulator. In *ACM SIGARCH Computer Architecture News*, volume 39, pages 1–7.
- Curso de Ciência da Computação – UFFS (2026). Avaliação de microarquitetura RISC-V com o simulador gem5. Trabalho 2 da disciplina Organização de Computadores.

Hennessy, J. L. and Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6th edition.

Lee, C.-C. and Chang, P. P. (1997). Bi-mode branch prediction. In *Proceedings of the 24th Annual International Symposium on Computer Architecture (ISCA)*, pages 4–13.

A. Comandos de compilação e simulação

```
# Compilar o binario estatico RISC-V
riscv64-linux-gnu-gcc -static heapSort_original.s \
    -o heapSort_original.riscv

# Simular no gem5 (CPU em-ordem, cenario padrao)
./gem5/build/RISCV/gem5.opt src/models/inorder.py \
    --binary src/heapSort_original.riscv

# Extrair metricas relevantes
grep -E "simTicks|ipc|branchPred" m5out/stats.json
```

B. Trecho de assembly com desvios críticos

```
check_left:
    ...
    blt t4, t6, set_left_min    # comparar filho esquerdo
    j check_right
check_swap:
    beq s3, a2, done_heapify    # precisa trocar?
    ...
    jal ra, heapify             # recursao
build_loop:
    blt s3, zero, build_done    # laço de construcao
extract_loop:
    ble s4, zero, extract_done  # laço de extracao
```